Defending Attack Graphs That Move: Telemetry-Driven Exposure, Incremental Shortest Paths, and Joint Detection–Deception Placement

Internal draft v0 — not for distribution

Lucas Sotomayor
BASE4 Security, R&D+i

November 4, 2026

Abstract

Attack-graph defenses for Active Directory (AD) are computed on snapshots, yet the edges that matter most—interactive sessions, local-admin grants, ACL changes—appear and expire within hours. We describe a program that treats the attack graph as a time-indexed state G_t maintained from security telemetry, and three components built on it. (i) A deterministic, idempotent *projection rule* that maps OCSF provenance events to attack-graph edge changes, with a conformance suite of 17 cases; writing it surfaced two defects in a production parser that we fixed. (ii) An *incremental oracle* that maintains shortest distances from every node to every crown jewel under edge insertions, deletions and weight changes, with a work-based fallback to full recomputation. On a synthetic AD graph with 4.6k nodes and 354k edges, frequent changes (sessions, admin grants) are repaired in 0.02–0.16 ms on average against 310 ms for recomputation; the worst case stays within 1.1× of recomputation. (iii) A *joint detection–deception* model that places detection rules on edges and traps on nodes under a per-path detection guarantee, obtained by a node-splitting reduction that makes traps ordinary rules for an existing exact MILP; it matches brute force on 100 instances with and without a trap budget. On 20 synthetic instances, two traps lower the median cost of a $\delta = 0.05$ guarantee to 11% of the rules-only optimum and raise feasibility from 6/20 to 18/20. All results are preliminary: synthetic catalogs, synthetic detection rates and no large real AD.

1 Introduction

Defensive analyses of AD attack graphs, from BloodHound path queries [22] to edge blocking [5, 6] and honeypot placement [13], take a collection snapshot as input. Two observations motivate this work. First, the edges that create short paths to Domain Admin are largely *transient*: a privileged user logging on to a workstation creates a `HasSession` edge that lasts as long as the credentials stay in memory. A snapshot taken on Monday says little about Thursday. Second, the telemetry that reveals those edges is already collected by the SOC—Windows 4624/4688, Sentinel, EDR—but it lives in a different graph: a provenance graph of what happened, not an attack graph of what could happen.

We connect the two. The attack graph becomes a state G_t updated by events, and every defensive question becomes a query or an optimisation over G_t.

Contributions.

- A two-layer model of G_t and a **projection rule** from OCSF provenance events to attack-graph changes, with explicit edge support, idempotent deltas, watermark-based expiry and identity resolution that refuses to guess (Section 4). A reference implementation passes a 17-case conformance suite shared with the production engine.
- An **incremental distance oracle** for crown-jewel exposure in the style of Ramalingam–Reps [18], with a fallback triggered by the *edge work* of the affected subtree rather than its size, which matters with group hubs (Section 5).
- A **joint detection–deception** formulation: a reduction that turns node traps into edge rules, so that one exact MILP chooses detection rules and traps under a per-path guarantee, plus a trap budget (Section 6).
- An evaluation of the three components on GOADv2 [16, 11] and synthetic AD graphs (Section 7), with every result verified against an independent reference (brute force or full Dijkstra).

2 Background and related work

Attack graphs and hardening. Attack graphs were formalised by Jha et al. [9]; minimum-cost hardening [24] and IDS sensor placement [14] are early optimisation

uses. For AD, Guo et al. study edge blocking with fixed-parameter [5] and scalable algorithms [6], and Goel et al. combine a neural dynamic program with evolutionary search [4].

Interdiction and security games. Shortest-path network interdiction [7] and its stochastic variants [12] are the operations-research root of our defender. Double-oracle methods solve zero-sum security games on graphs [8]. Alert triage as a game is studied by Schlenker et al. [19]. Arc interdiction combined with monitoring appears in Nguyen, Song and Smith [**TODO:** full reference, Networks 2023].

Deception. Honeypot allocation over attack graphs [1], the survey by Pawlick et al. [17], and honeypot placement in dynamic AD graphs [13] are closest to our trap model. Ngo et al. also treat the graph as dynamic, but the dynamics are a model of the attacker's moves, not a stream of observed telemetry.

Dynamic shortest paths. Incremental single-source shortest paths go back to Ramalingam and Reps [18]; LPA* [10] is the planning variant; Demetrescu and Italiano [3] give fully dynamic all-pairs bounds. We use the simplest of these ideas and show where it pays off on AD graphs and where it does not.

Telemetry schemas. OCSF [15] is the event schema of the projection; detection content is mapped to ATT&CK techniques, whose use in EDR products was measured by Virkud et al. [23].

3 The state G_t

Definition 1 (Two-layer state)**.** $G_t = (V_t,\ E_t^{\text{atk}} \cup E_t^{\text{prov}},\ w_t,\ q_t,\ r)$. *Attack edges* $(u, \kappa, v) \in E_t^{\text{atk}}$ *mean "controlling* u *lets the attacker acquire* v*" with BloodHound semantics and directions [22]:* `HasSession` *goes from host to user,* `AdminTo` *and* `CanRDP` *from principal to host. Provenance edges are timestamped OCSF events.* q_t *are detection probabilities on edges (rules) or nodes (traps),* w_t *are attack-edge weights and* r *node values.*

Each attack edge carries its *origins* (`snapshot`, `observed`), its *support* (the set of event ids sustaining it), first and last evidence times, an expiry time and a confidence (`confirmed` if an event shows it exercised, `inferred` if it is only enabled). An edge exists in G_t while it has the snapshot origin, or a non-empty support and an unexpired expiry. This keeps both sources of truth: the collector's snapshot and what telemetry has observed since.

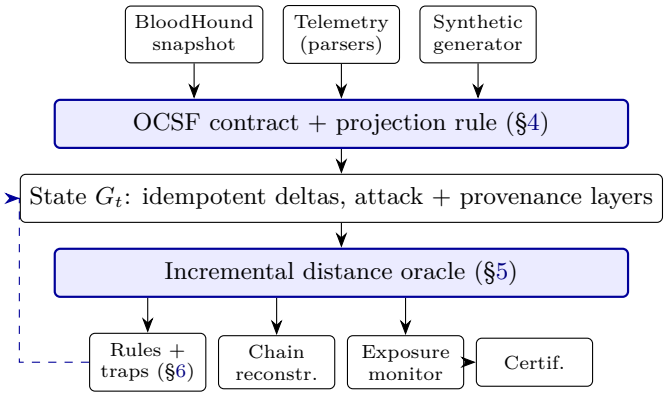


Figure 1: Architecture. Shaded components are new in this work. The dashed arrow is the feedback loop: what the defender deploys enters G_t as deltas.

4 Projection: provenance to attack edges

The projection is a pure, deterministic function from an OCSF envelope and an identity table to a list of `AttackDeltas`: Assert(edge, event, confidence, ttl), Retract(edge, event), Close(edge, at) and Unresolved(reason). Deltas are keyed by event id, so replaying an envelope is a no-op; a tombstone retracts the support its event contributed; a correction is a retract followed by re-projection.

Design choices. Interactive, remote-interactive and cached logons leave reusable credentials on the host, so they are sessions in the BloodHound sense; network logons (type 3) normally do not, and become `AdminTo` evidence only together with privilege events that the engine does not parse yet. A logoff does not erase credentials from memory immediately, so it closes the session after a grace period rather than retracting it.

Time. The time of an edge is the event time. An out-of-order event can lower *first seen* but never advances expiry, and expiry is driven by the stream watermark, not the wall clock: reprocessing a historical day yields the same state as processing it live.

Identity resolution. Parsers name users `NETBIOS\user` and hosts by whatever the event carries; BloodHound uses UPN-like names, FQDNs and SIDs. The rule uses the SID when present, otherwise normalises against the snapshot's domain table and accepts a short host name only if exactly one FQDN matches. Otherwise it emits Unresolved: a session on the wrong host invents an attack path, which is worse than a missing one.

Table 1: Projection rules v0. lt is the OCSF `logon_type_id`, numbered as the Windows 4624 LogonType for values 2–13.[**TODO:** verify against the official OCSF 1.4 enum]

Event	Condition	Output
Auth. logon	success, $lt \in \{2, 10, 11, 12\}$	`HasSession`($h \to u$), confirmed
Auth. logon	success, $lt = 10$	also `CanRDP`($u \to h$)
Auth. logon	success, $lt \in \{4, 5, 9\}$	`HasSession`, inferred
Auth. logon	$lt \in \{3, 8\}$, failure, no lt	nothing
Auth. logoff	session observed	CLOSE with grace
Process launch	user not system/machine	`HasSession`, inferred, short ttl
Tombstone	—	RETRACT its support

Algorithm 1: Repair after deleting or raising edge (u, v) for terminal t

if $\sigma_t(u) \neq v$ **then**
  return ; `// not a tree edge`
$A \leftarrow \{u\}$; $W \leftarrow \deg(u)$;
foreach x *popped from a stack started at* u **do**
  foreach *in-neighbour* p *of* x *with* $\sigma_t(p) = x$, $p \notin A$ **do**
    $A \leftarrow A \cup \{p\}$; $W \leftarrow W + \deg(p)$;
    if $W > \tau\,|E|$ **then** recompute t from scratch;
     return;
foreach $x \in A$ **do** $d_t(x) \leftarrow \min_{(x,y),\, y \notin A} w(x, y) + d_t(y)$;
run Dijkstra on the reverse graph restricted to A, seeded with those values;

Defects found. Specifying the rule against the production engine exposed six prerequisites; two were defects. The OCSF projection dropped the logon type, and a failed 4625 parsed by the Sysmon/Security parser reached OCSF without a status—indistinguishable from a successful logon—because that parser flagged failures with a field the projection did not read. Both are fixed upstream with tests; the other four (logoff, privilege and group-change events, SIDs, process host) are missing data, not errors.

5 Incremental distance oracle

For each terminal t (a crown jewel), the oracle keeps $d_t(v)$, the shortest distance from v to t under non-negative weights, and the next hop $\sigma_t(v)$. One structure per terminal answers "how close is any compromised node to t", and distances between terminals give the matrix that routing-style solvers need.

Insertions and decreases propagate the improvement backwards with Dijkstra and touch only nodes that improve. Deletions and increases (Algorithm 1) do nothing unless the edge is in the shortest-path tree; otherwise only the subtree A of nodes whose path went through it is reset, seeded from the unaffected frontier, and repaired.

Proposition 1. *After Algorithm 1, d_t equals the shortest distances in the updated graph.*

Sketch. A node outside A keeps a tree path to t that avoids (u, v); its length is unchanged and, since the update only removes options, no shorter path appeared. Nodes in A are recomputed by Dijkstra from correct boundary values, which is exact because every shortest path from $x \in A$ leaves A at its first node outside A. □

Work-based fallback. The first version fell back to recomputation when $|A|$ exceeded a fraction of $|V|$. On AD graphs with expanded groups this was the wrong measure: a few hundred nodes can carry hundreds of thousands of edges, and local repair became slower than recomputation. The fallback now counts the in- and out-degree of the nodes added to A and stops as soon as it exceeds $\tau|E|$ ($\tau = 0.1$).

What-if. `what_if(changes)` applies a batch, yields, and restores the previous weights; selection loops that price one candidate rule or trap at a time use it.

6 Joint detection and deception

Detection guarantee. Following detcover [20], a rule r covers the edges whose ATT&CK technique and telemetry scope it observes, fires with probability q_r and costs c_r expected false positives per day. Under independence, the probability that a path P evades the active rules x is $\prod_{e \in P} \prod_{r \ni e} (1 - q_r)^{x_r}$, so requiring every source–terminal path to be detected with probability at least $1 - \delta$ is linear in x through potentials:

$$\begin{aligned} \min\ & \textstyle\sum_r c_r x_r \\ \text{s.t.}\ & \pi_v - \pi_u - \textstyle\sum_{r \ni e} w_r x_r \leq 0 \quad \forall e = (u, v), \\ & \pi_s = 0,\ \pi_t \geq \theta,\ 0 \leq \pi \leq \theta,\ x \in \{0, 1\}^R, \end{aligned}$$

with $w_r = -\log(1 - q_r)$ and $\theta = -\log \delta$. The same codebase solves a variant with edge blocking, a randomised triage defender against an informed attacker (Stackelberg), and a lazy constraint-generation version for large graphs.

Traps as rules. PERIPLO [21] places traps on nodes: acquiring a trapped node is detected with certainty. We split every trap candidate v (never the source or a terminal)

into $v \to v^{\#}$, redirect v's out-edges to leave from $v^{\#}$, and add a rule `trap:`v with $q = 1$ that covers only the new edge.

Proposition 2. *For any choice of rules R' and traps T', the minimum evasion weight from s to the terminals in the split instance with rules $R' \cup \{\texttt{trap:}v : v \in T'\}$ equals the one in the original instance with rules R' on edges and traps T' on nodes.*

Sketch. Paths are in bijection: a path that continues from v must traverse $v \to v^{\#}$, which carries exactly the trap weight when $v \in T'$ and zero otherwise; edge weights of redirected edges are unchanged because coverage depends only on technique and scope. Terminals are not split, and a path that ends at v is not a source–terminal path. □

Consequently every detcover solver chooses rules and traps jointly without modification. A trap budget, as in PERIPLO, is one extra row $\sum_v x_{\texttt{trap:}v} \le k$.

Scope of the equivalence. detcover's attacker wants to reach a terminal, so a trap matters only on transit nodes. PERIPLO's attacker collects value per node under a time budget and a pivot-switch cost; a trap on a dead-end node matters to it and not to this reduction. Evaluating joint solutions against PERIPLO's attacker is future work.

7 Evaluation

Everything runs in Python on one core of a shared cloud VM; times are single runs and vary between runs by up to ∼1.5× for full recomputation. Code and data live in the Graph Hunter repository [2] under `research/` (private branch).

7.1 Projection

The reference implementation passes the 17 conformance cases (all rules, idempotence, out-of-order arrival, tombstones, corrections, ambiguous hosts and unknown domains). The same JSON file is the target for the Rust implementation. The parser and schema fix adds four tests; 57 tests of the canonical crate and 1,033 of the core engine pass.

7.2 Oracle

Correctness. After every change, distances and paths are compared with a full Dijkstra: 4,800 random changes on 30-node graphs (with and without the fallback), 5×200 on GOADv2 and 150 on a synthetic AD graph with hubs. Disabling the repair of increases on purpose makes the tests fail.

Table 2: Oracle on a synthetic AD graph (`adgen`, scale 2000): 4,583 nodes, 353,526 edges, 4 terminals. Full recomputation for the 4 terminals: 310 ms. Each change is measured and then undone; "fallbacks" counts recomputations from scratch.

Change	mean (ms)	worst (ms)	fallbacks	speed-up
New session	0.025	0.36	0	12,557×
Session expires	0.161	2.27	0	1,924×
AdminTo removed	0.024	0.48	0	12,782×
ACL removed	2.31	226	3	134×
Tree edge near t	20.2	346	21	15×

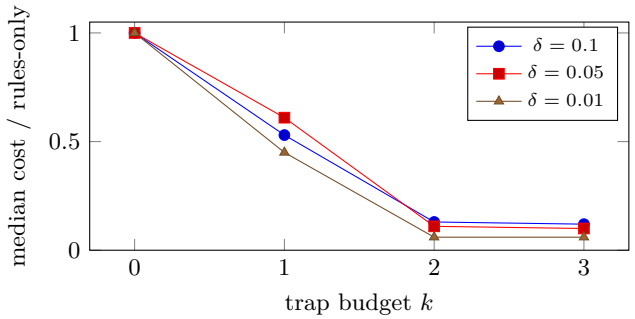


Figure 2: Cost of the guarantee with up to k traps (trap cost 5 FP/day-equivalent), median over the instances feasible with rules only. 20 synthetic instances.

Cost. Table 2 reports the graph whose group expansion creates hubs. Frequent changes cost thousands of times less than recomputation because most of them do not touch any shortest-path tree. The adversarial case, removing a tree edge next to a terminal, averages 15× faster and its worst case (346 ms) stays within 1.1× of recomputation thanks to the work-based fallback; before the fallback was changed, local repair exceeded recomputation. On a sparser random AD-like graph (62,855 nodes, 132,507 edges) the mean repair is 0.016 ms against 199 ms per terminal for recomputation. GOADv2 (46 nodes, 259 edges) is too small to say anything about scale; all its changes take under 0.14 ms.

7.3 Joint detection and deception

Exactness. On 25 random instances (9 nodes, 5 rules, 3 trap candidates) the MILP optimum on the split instance equals a brute force over rules and node traps that does not use the reduction; the cases include optima with traps (8), without (7), and infeasible instances (10). With a budget $k \in \{0, 1, 2\}$ the same holds on 75 more instances. For random choices of rules and traps, evasion distances coincide.

Synthetic sweep. On 20 instances of detcover's generator (121 nodes, about 340 edges, 25 rules with synthetic detection rates), the best evasion probability reachable with all rules active has median 0.076, so rules alone can-

Table 3: Feasible instances out of 20 (trap cost 5).

δ	$k=0$	$k=1$	$k=2$	$k=3$
0.10	12	17	18	18
0.05	6	14	18	18
0.01	1	7	12	17

not guarantee $\delta = 0.05$ in 14 of 20 instances (Table 3). Two traps restore feasibility in 18 of 20 and cut the median cost to 11% of the rules-only optimum where both exist (Figure 2); at a trap cost of 15 the reduction is to 25%. Every MILP solves in under 0.2 s.

GOADv2. Using as footholds the users PERIPLO classifies as unprivileged, the best evasion probability with every rule of the (synthetic) catalog is 0.12 in ESSOS, 0.33 in NORTH and 0.54 in SEVENKINGDOMS. One trap on a transit user or computer gives the $\delta = 0.05$ guarantee in ESSOS and NORTH; in SEVENKINGDOMS the six admissible trap candidates do not suffice. With detcover's default of 20 random footholds, GOAD's small domains include accounts that are already Domain Admins and every instance is infeasible.

8 Discussion and limitations

- **Traps are perfect by assumption.** The reduction inherits PERIPLO's $q = 1$; one trap then dominates any number of imperfect rules. A realistic trap model needs an interaction probability, which the reduction already supports through the rule's q.
- **Synthetic detection content.** Catalogs and detection rates are synthetic; detcover already imports Sigma rules, which is the next step.
- **Independence.** The linear guarantee assumes independent firings; detcover quantifies the effect of common-cause correlation separately.
- **No large real AD.** GOADv2 is small; client collections cannot enter the repository. The worst case of the oracle on real hubs is measured only on a synthetic generator.
- **Projection parameters.** Session lifetimes and grace periods are uncalibrated defaults; four prerequisites remain open (logoff, privilege and group-change events, SIDs, process host).

9 Roadmap

(1) Implement the projection in the Rust engine against the same conformance file. (2) Evaluate joint solutions against PERIPLO's value-collecting attacker and its exact best response, and unify the certificate formats of the solvers. (3) Calibrate edge weights from attack chains reconstructed by the engine's Steiner-arborescence module, closing the loop from observed intrusions to model parameters. (4) Measure the oracle and the joint model on a large real collection under the appropriate data agreements.

Items to verify before submission

- Full reference for Nguyen, Song and Smith (Networks, 2023).
- OCSF 1.4 `logon_type_id` values (checked only against a secondary source).
- Page range of [8]: the two source bibliographies disagree (305–312 vs 327–334).
- Ramalingam–Reps, LPA* and Demetrescu–Italiano entries were written from memory.

References

[1] Ahmed H. Anwar, Charles Kamhoua, and Nandi Leslie. Honeypot allocation over attack graphs in cyber deception games. In *2020 International Conference on Computing, Networking and Communications (ICNC)*, pages 502–506. IEEE, 2020.

[2] BASE4 Security. Graph Hunter: Graph-based and hypothesis-driven threat hunting. https://github.com/Base4Security/GraphHunter, 2026.

[3] Camil Demetrescu and Giuseppe F. Italiano. A new approach to dynamic all pairs shortest paths. *Journal of the ACM*, 51(6):968–992, 2004.

[4] Diksha Goel, Max Ward, Aneta Neumann, Frank Neumann, Hung Nguyen, and Mingyu Guo. Defending Active Directory by combining neural network based dynamic program and evolutionary diversity optimisation. In J. E. Fieldsend and M. Wagner, editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '22)*, pages 1191–1199, New York, NY, 2022. ACM.

[5] Mingyu Guo, Jialiang Li, Aneta Neumann, Frank Neumann, and Hung Nguyen. Practical fixed-parameter algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9360–9367, 2022.

[6] Mingyu Guo, Max Ward, Aneta Neumann, Frank Neumann, and Hung Nguyen. Scalable edge blocking algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 5649–5656, 2023.

[7] Eitan Israeli and R. Kevin Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002.

[8] Manish Jain, Dmytro Korzhyk, Ondřej Vaněk, Vincent Conitzer, Michal Pěchouček, and Milind Tambe. A double oracle algorithm for zero-sum security games on graphs. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2011)*, pages 327–334, 2011.

[9] Somesh Jha, Oleg Sheyner, and Jeannette M. Wing. Two formal analyses of attack graphs. In *Proceedings of the 15th IEEE Computer Security Foundations Workshop (CSFW)*, pages 49–63, 2002.

[10] Sven Koenig, Maxim Likhachev, and David Furcy. Lifelong planning A*. *Artificial Intelligence*, 155(1–2):93–146, 2004.

[11] m4lwhere. Bloodhound-CE-Sample-Data: Sample data to load into BloodHound Community Edition, based on GOADv2. https://github.com/m4lwhere/Bloodhound-CE-Sample-Data. SharpHound v2.3.3 collection. Accessed 2026-10-22.

[12] David P. Morton, Feng Pan, and Kevin J. Saeger. Models for nuclear smuggling interdiction. *IIE Transactions*, 39(1):3–14, 2007.

[13] Huy Q. Ngo, Mingyu Guo, and Hung Nguyen. Catch me if you can: Effective honeypot placement in dynamic AD attack graphs. In *IEEE INFOCOM 2024 – IEEE Conference on Computer Communications*, pages 451–460. IEEE, 2024.

[14] Steven Noel and Sushil Jajodia. Optimal IDS sensor placement and alert prioritization using attack graphs. *Journal of Network and Systems Management*, 2008.

[15] OCSF Project. Open cybersecurity schema framework, version 1.4. https://schema.ocsf.io.

[16] Orange Cyberdefense. GOAD: Game of Active Directory. https://github.com/Orange-Cyberdefense/GOAD. Accessed 2026-10-22.

[17] Jeffrey Pawlick, Edward Colbert, and Quanyan Zhu. A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy. *ACM Computing Surveys*, 52(4):1–28, 2019.

[18] G. Ramalingam and Thomas Reps. An incremental algorithm for a generalization of the shortest-path problem. *Journal of Algorithms*, 21(2):267–305, 1996.

[19] Aaron Schlenker et al. Don't bury your head in warnings: A game-theoretic approach for intelligent allocation of cyber-security alerts. In *Proceedings of IJCAI*, pages 381–387, 2017.

[20] Lucas Sotomayor. detcover: Minimum-cost detection cuts on attack graphs, 2026. Unpublished code and manuscript, commit 70b6a13.

[21] Lucas Sotomayor. PERIPLO: Certified randomized deception against multi-target attackers in Active Directory attack graphs. https://github.com/sotomayorlucas/periplo, 2026. Code and manuscript, commit 1456ec1.

[22] SpecterOps. BloodHound: Six Degrees of Domain Admin. https://github.com/SpecterOps/BloodHound. Accessed 2026-10-22.

[23] Apurva Virkud, Muhammad Adil Inam, Andy Riddle, Jason Liu, Gang Wang, and Adam Bates. How does endpoint detection use the MITRE ATT&CK framework? In *Proceedings of the 33rd USENIX Security Symposium*, pages 3891–3908, 2024.

[24] Lingyu Wang, Steven Noel, and Sushil Jajodia. Minimum-cost network hardening using attack graphs. *Computer Communications*, 2006.